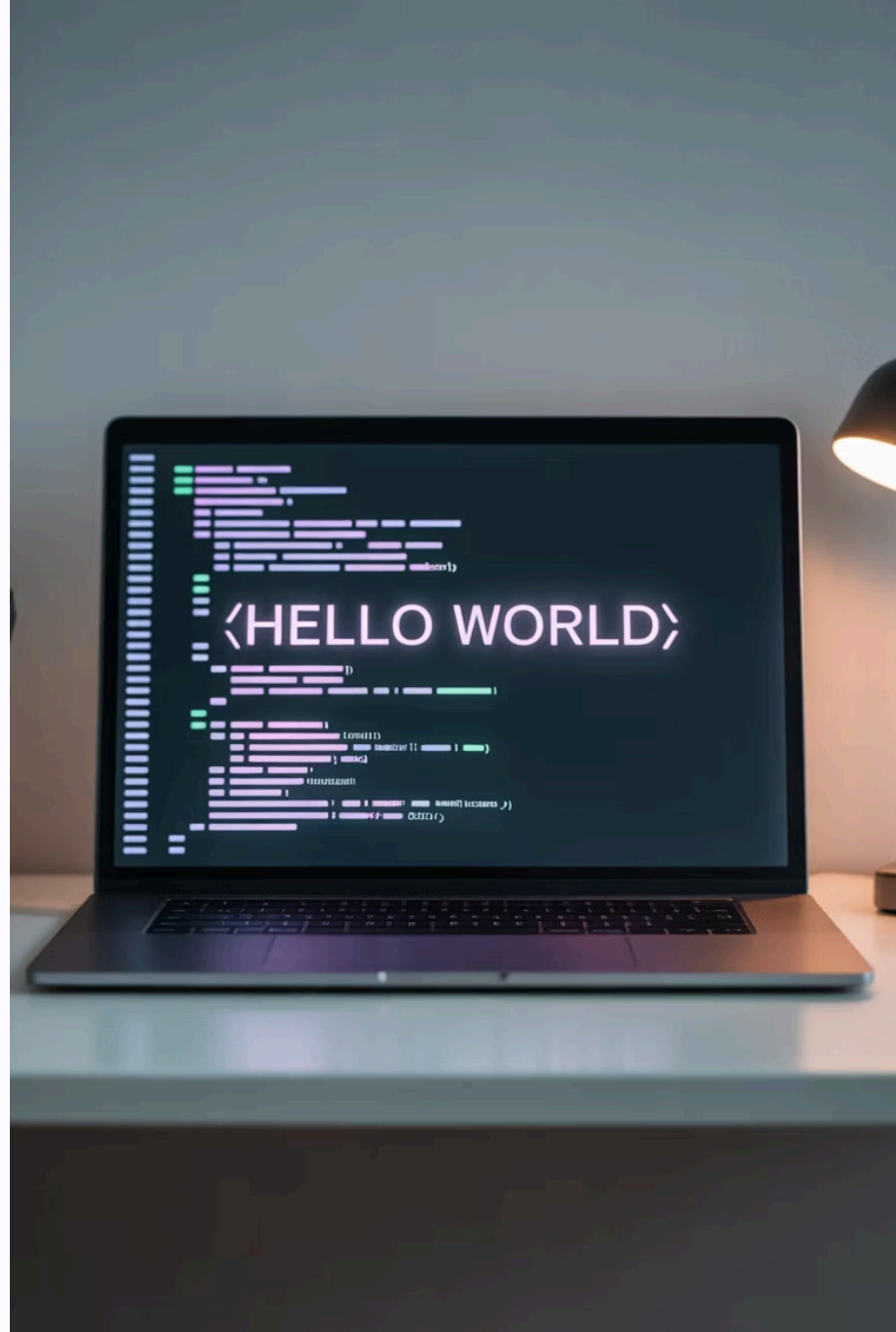


Condicionales y Bucles en JavaScript

Los condicionales y bucles son estructuras fundamentales que controlan el **flujo de ejecución** en JavaScript. Estas herramientas permiten que nuestros programas tomen decisiones inteligentes y repitan tareas de manera eficiente.

Los **condicionales** ejecutan código cuando se cumplen ciertas condiciones, mientras que los **bucles** repiten instrucciones hasta que una condición cambie o se complete una iteración sobre datos.



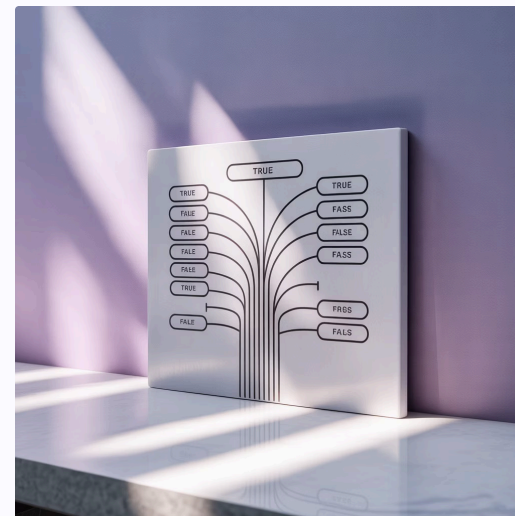
Condicional if y else

La estructura `if...else` es la base de la **toma de decisiones** en programación. Evalúa una condición y ejecuta diferentes bloques de código según el resultado.

Sintaxis básica:

```
if (condición) {  
    // Código si es verdadero  
} else {  
    // Código si es falso  
}
```

La condición debe ser una expresión que devuelva `true` o `false`. JavaScript también convierte automáticamente otros valores a booleanos.



Ejemplo práctico:

```
let edad = 18  
if(edad >= 18){  
    console.log("Sos mayor!")  
}else{  
    console.log("Sos menor!")  
}
```

Condicional else if

Cuando necesitamos evaluar **múltiples condiciones** en secuencia, utilizamos `else if`. Esta estructura permite crear cadenas de decisiones más complejas y específicas.

01

Primera condición

Se evalúa la condición del `if` inicial

02

Condiciones alternativas

Si la primera falla, se evalúan los `else if` en orden

03

Opción por defecto

El `else` final se ejecuta si ninguna condición es verdadera

Ejemplo con calificaciones:

```
let nota = 7;
if (nota >= 9) {
  console.log("Excelente");
} else if (nota >= 6) {
  console.log("Aprobado");
} else {
  console.log("Reprobado");
}
```

A decorative header image featuring a neon-style graphic of a switch. The word "SWITCH" is written in a glowing, outlined font in the center. To its left, a blue neon line enters from the top left and turns right into the switch. To its right, a red neon line enters from the top right and turns left into the switch. Above the switch, the text "CASE 1" is visible in a glowing outline, and below it, "CASE 2" is partially visible. The background is a dark, starry space with some nebulae.

Condicional Switch

El statement `switch` es ideal cuando tenemos **múltiples opciones** basadas en el valor de una sola variable. Es más limpio y legible que largas cadenas de `else if`.

Características clave:

- Compara usando igualdad estricta (`===`)
- Requiere `break` para evitar "fall-through"
- `default` maneja casos no contemplados
- Más eficiente para muchas opciones

```
let dia = 3;
switch (dia) {
  case 1:
    console.log("Lunes");
    break;
  case 2:
    console.log("Martes");
    break;
  case 3:
    console.log("Miércoles");
    break;
  default:
    console.log("Día inválido");
}
```

Bucle for

El bucle `for` es perfecto cuando sabemos **exactamente cuántas veces** queremos repetir una operación. Su estructura compacta incluye inicialización, condición y actualización en una sola línea.



Inicialización

```
let i = 0
```

Declara y asigna valor inicial al contador



Condición

```
i < 5
```

Se evalúa antes de cada iteración



Actualización

```
i++
```

Se ejecuta después de cada iteración

Ejemplo básico:

```
for (let i = 0; i < 5; i++) {  
  console.log("Iteración: " + i);  
}  
// Salida: 0, 1, 2, 3, 4
```

Bucle while



El bucle **while** continúa ejecutándose **mientras la condición sea verdadera**. Es útil cuando no sabemos exactamente cuántas iteraciones necesitaremos.

⚠ **¡Cuidado!** Asegúrate de que la condición eventualmente se vuelva falsa, o crearás un bucle infinito.

Estructura y ejemplo:

```
let n = 1;
while (n <= 3) {
  console.log("Número: " + n);
  n++; // ¡Importante!
}
```

Este bucle imprime números del 1 al 3. La variable **n** debe incrementarse dentro del bucle para que la condición eventualmente sea falsa.

Casos de uso: validación de entrada del usuario, procesamiento de datos hasta completar, búsqueda con condiciones dinámicas.

Bucle do...while

La diferencia clave del `do...while` es que se ejecuta **al menos una vez** antes de evaluar la condición. Esto es útil cuando necesitamos que el código se ejecute una vez sin importar las condiciones iniciales.

while

Evalúa condición **antes** de ejecutar

Puede no ejecutarse nunca

do...while

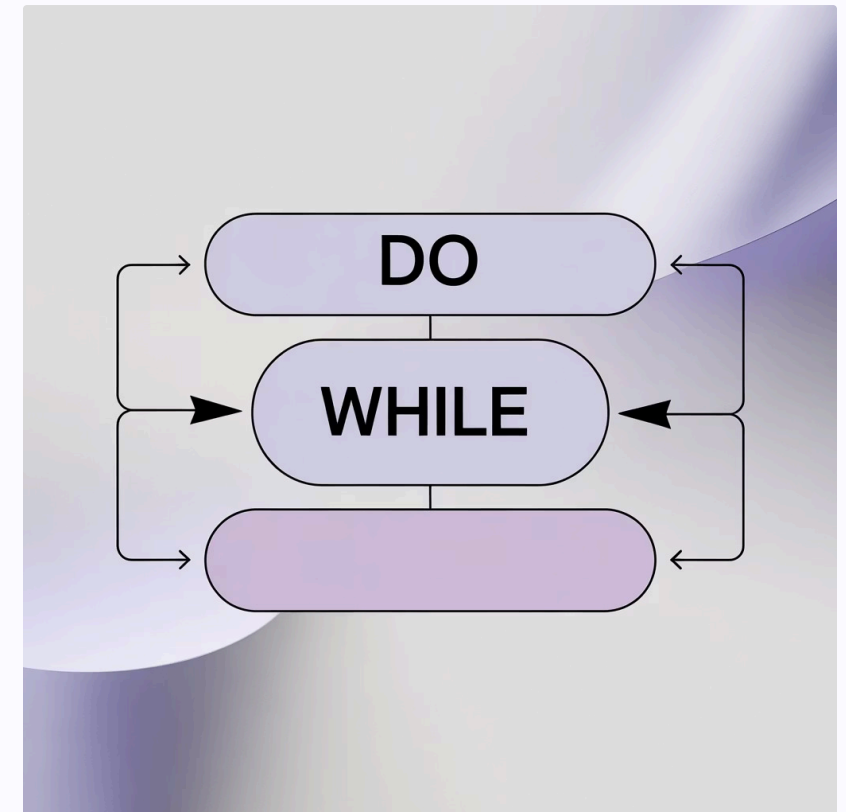
Ejecuta **primero**, luego evalúa condición

Se ejecuta al menos una vez

Ejemplo práctico:

```
let x = 1;  
do {  
  console.log("Ejecutado: " + x);  
  x++;  
} while (x <= 3);
```

Ideal para menús interactivos, validación de entrada del usuario, o cualquier proceso que debe ocurrir al menos una vez.



Bucle for...of

El bucle `for...of` está diseñado específicamente para **recorrer elementos** de arrays, strings y otros objetos iterables. Es más simple y legible que un bucle `for` tradicional para estas tareas.



Arrays

Recorre cada elemento del array sin necesidad de índices



Strings

Itera sobre cada carácter de una cadena de texto

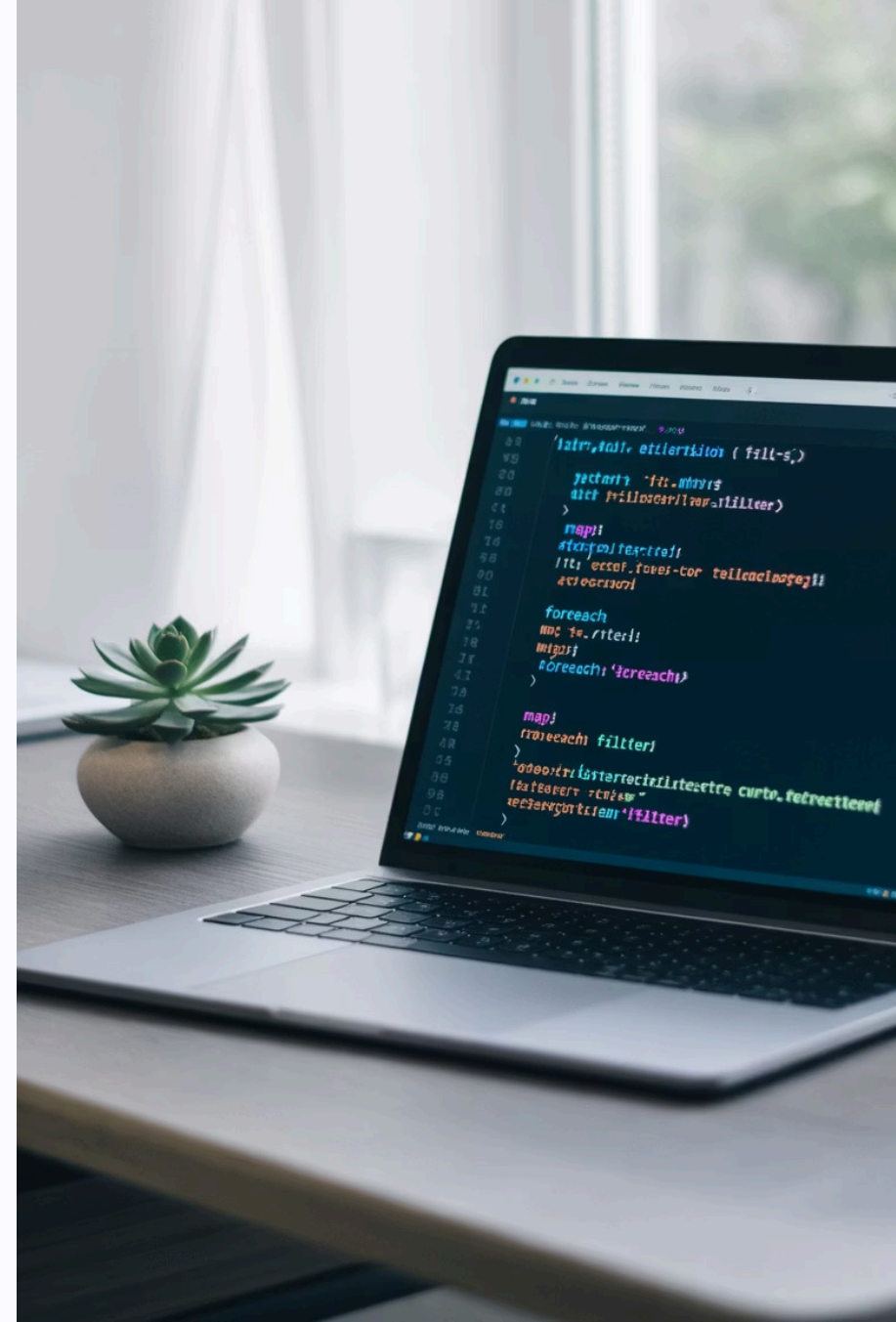


Iterables

Funciona con Set, Map y otros objetos iterables

Ejemplo con array de frutas:

```
let frutas = ["manzana", "banana", "uva"]
```



Método forEach

El método `forEach` es una función específica de los arrays que ejecuta una función callback para cada elemento. Es una alternativa moderna y funcional a los bucles tradicionales.

Sintaxis y parámetros:

```
array.forEach((elemento, indice, array) => {  
  // Código a ejecutar  
});
```

- **elemento:** el valor actual del array
- **índice:** posición del elemento (opcional)
- **array:** referencia al array original (opcional)



Ejemplo práctico:

```
let numeros = [1, 2, 3];  
numeros.forEach((num, i) => {  
  console.log(`Posición ${i}: ${num * 2}`);  
});
```

📌 Ventaja: `forEach` es más expresivo y funcional. **Limitación:** no se puede usar `break` o `continue` como en bucles tradicionales.

Resumen: Control de Flujo en JavaScript

Dominar estas estructuras de control es esencial para crear programas dinámicos e interactivos. Cada una tiene su propósito específico en diferentes situaciones de programación.



Condicionales

if/else y switch para tomar decisiones basadas en condiciones



Bucles Tradicionales

for, while, do...while para repetir operaciones



Iteración de Arrays

for...of y forEach para recorrer colecciones de datos

¡Practica con estos conceptos para dominar el **control de flujo** en JavaScript!

